

Case Study (Data Analysis and Web App)

Explain every important step clearly. Where useful, use a practical scenario, visualizations, or other supporting material.

1. Project Setup and Data Import

- Create a Jupyter Notebook for the case study.
- Add the group number and names of all participants.
- Download original dataset from the tubCloud dataset.
- Write and describe the task, your interpretation, and your data-selection strategy.
- List and import the required files.
- If you first inspect or select files manually, explain what you did and why.
- Show with code how you identified the relevant files, components, or IDs.
- Perform exploratory data analysis using comments and suitable visualizations.
- Document every step in the notebook.

2. Data Preparation

- Inspect and clean the data using the methods taught in the course.
- Follow tidy-data principles.
- Create new features where necessary.
- Combine all relevant data into one final dataset.

3. Develop the App

- Use packages compatible with the current Python version.
- The app must run directly from the submission folder on another computer without changes.
- Test this before submission.
- The app must use only the final dataset created in Step 2.
- Design the app for the intended target group.
- Use light blue as the main colour.
- Add your own logo or the Department of Quality Science logo.
- Use the **Source Sans Pro** font.

4. App Content and Visualizations

- Select information that is useful for the relevant stakeholders.
- If the task does not specify particular results, present other meaningful findings.
- Add useful interactive features, such as filters, maps, or selectable charts.

- Present the results via App like a real case.
- Include a table page showing all final data in App.

5. Document the Results

- Explain the analysis process step by step with meaningful visualizations.
- Discuss the results and refer to the interactive charts in the app.
- Comment the code clearly and follow the PEP 8 style guide.
- Clear documentation may receive partial credit even if some code does not work completely. Unclear documentation may cause decrease in point even the step exists.
- Include meaningful screenshots of the app in the results section.

Information about Data

All information, including when a vehicle was produced in which plant and whether a vehicle is defective, can be found in the group's production data. **A vehicle is always considered to be defective if an installed individual part, an installed component, or the entire vehicle is marked as defective.** This logic also applies accordingly to components that contain defective individual parts.

You are given access to the group's own database and must decide for yourself which data you require for your analysis. Furthermore, the German Federal Motor Transport Authority (KBA) has provided registration data and geodata, which are also stored in the database. The categories of datasets are listed below:

- Single Part = **Einzelteil**
- Component = **Komponente**
- Vehicle = **Fahrzeug**
- Geodata = **Geodaten**
- Registrations = **Zulassungen**
- Logistics delay = **Logistikverzug**

It is recommended to closely examine the structure of the respective tables before starting the analysis.

For all instances of the supply chain, i.e., for single parts, components, and vehicles, production data with information on ID number, manufacturer, manufacturing plant, production date, and error entries are available. The ID numbers consist of part designation, manufacturer, plant, and sequential number. Example: 1-201-2011-3, component T1, produced by manufacturer "201" in plant "2011", 3rd component from this series.

For components and vehicles, there are also parts lists available, which are declared by the naming convention Components_Name_Abbreviation. They contain information about all parts or components installed in each case.

Submission Deadline: 1 September 2026, 23:59.